Experiment 9

Aim : Implement Recurrent Neural Network (RNN) / LSTM for time series or text data

Dataset : time series - time_series_data.npy

Theory :

Recurrent Neural Networks (RNNs) are a class of neural networks designed to handle sequential data, such as time series or text. Unlike standard networks, RNNs have "memory" as they take information from prior inputs to influence the current input and output.

1. The Problem with Standard RNNs Standard RNNs struggle to remember information for long periods due to a mathematical issue called the "Vanishing Gradient" problem. As the sequence gets longer, the network effectively "forgets" the beginning of the sequence.

2. Long Short-Term Memory (LSTM) LSTMs are a special kind of RNN specifically designed to avoid the long-term dependency problem. They can remember information for long periods of time.

- **The Memory Cell:** Think of this as a conveyor belt that runs through the entire sequence, carrying relevant information.
- **Gates:** LSTMs use three "gates" to control the flow of information:
    - **Forget Gate:** Decides what information from the past is no longer useful and should be thrown away.
    - **Input Gate:** Decides which new information is worth adding to the memory cell.
    - **Output Gate:** Decides what parts of the memory cell should be output as the prediction for the current step.

3. Time Series Preprocessing (The Look-Back Method) Computers cannot simply "look" at a line; they need structured data. We use a sliding window approach called **Look-Back**.

- If we set a look-back of 10, the model looks at the values from time steps 1 to 10 to predict the value at step 11.
- The window then slides forward one step (looking at 2–11 to predict 12), and so on. This turns a single list of numbers into a set of training features.

4. Model Architecture

- **LSTM Layer:** We used 50 units. More units allow the model to learn more complex patterns but require more computing power.
- **Dense Layer:** Since we are predicting a single numerical value (regression), the final layer has 1 unit with a linear output.
- **Loss Function (MSE):** Mean Squared Error is used because it heavily penalizes large errors, which is ideal for precise time series forecasting.

Dataset Theory(Time Series): For this experiment, a synthetic sine wave was used. Time series data is unique because the order of the data points matters. If you shuffle the data, the patterns disappear. This is why we split the data sequentially (taking the first 80% for training and the last 20% for testing) rather than using a random shuffle.

Code and Outputs :

Step 1 : Input data and Preprocessing

```
import numpy as np

# 1. Define a function to create sequences
def create_sequences(data, look_back):
    X, y = [], []
    for i in range(len(data) - look_back):
        X.append(data[i:(i + look_back)])
        y.append(data[i + look_back])
    return np.array(X), np.array(y)

# 2. Choose a look-back value and apply the function
look_back = 10 # or 20, as suggested. Let's start with 10.
X, y = create_sequences(time_series_data, look_back)

# 3. Split the data into training and testing sets (80% train, 20% test)
train_size = int(len(X) * 0.8)
X_train, X_test = X[0:train_size], X[train_size:len(X)]
y_train, y_test = y[0:train_size], y[train_size:len(y)]

# 4. Reshape X_train and X_test for LSTM input (samples, time_steps, features)
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

print("Shape of X_train:", X_train.shape)
print("Shape of y_train:", y_train.shape)
print("Shape of X_test:", X_test.shape)
print("Shape of y_test:", y_test.shape)
```

Output :

```
Shape of X_train: (792, 10, 1)
Shape of y_train: (792,)
Shape of X_test: (198, 10, 1)
Shape of y_test: (198,)
```

Step 2 : Build and Compile RNN , using LSTM

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# 2. Create a Sequential model
model = Sequential()

# 3. Add an LSTM layer
# The input_shape should be (time_steps, features).
# In our case, time_steps is 'look_back' and features is 1 (univariate time series).
model.add(LSTM(units=50, activation='relu', input_shape=(look_back, 1)))

# 4. Add one or more Dense layers
model.add(Dense(units=1)) # Output layer for predicting the next single value

# 5. Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

# 6. Print the model summary
print(model.summary())
```

Output :

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 50)	10,400
dense (Dense)	(None, 1)	51

```
Total params: 10,451 (40.82 KB)
Trainable params: 10,451 (40.82 KB)
Non-trainable params: 0 (0.00 B)
None
```

Step 3 : Train the model

```
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_data=(X_test, y_test))

print("Model training complete.")
```

Output :

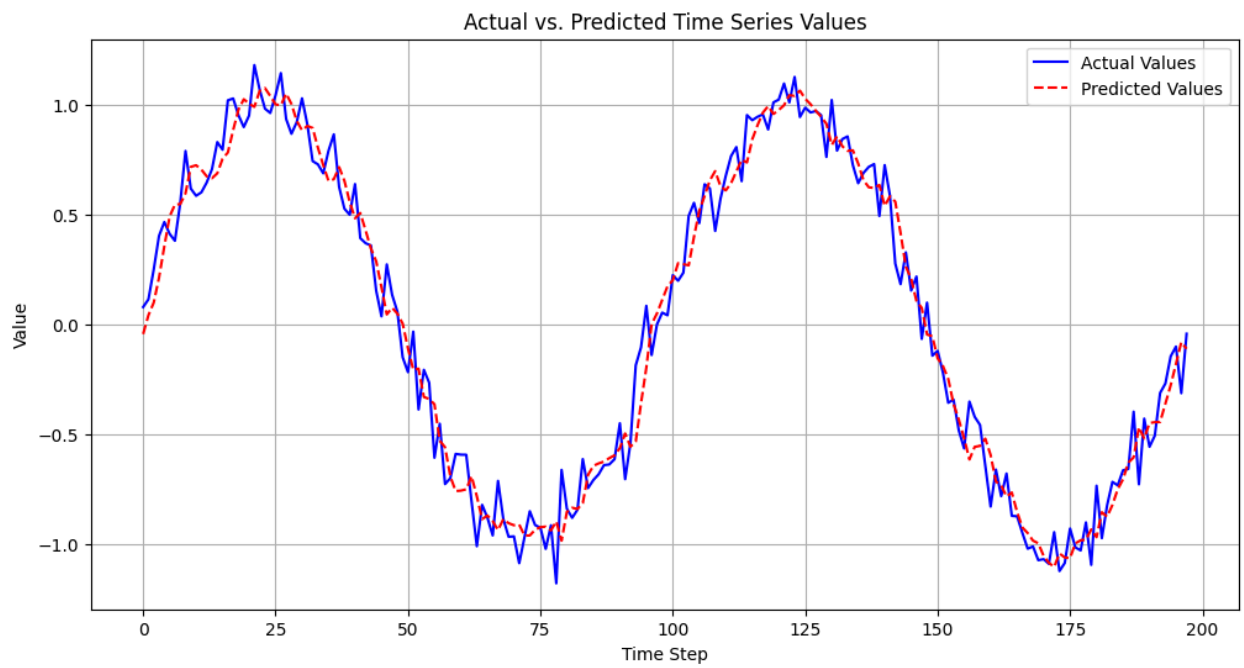
```
Epoch 45/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0146 - val_loss: 0.0160
Epoch 46/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0148 - val_loss: 0.0159
Epoch 47/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0149 - val_loss: 0.0170
Epoch 48/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0157 - val_loss: 0.0161
Epoch 49/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.0150 - val_loss: 0.0158
Epoch 50/50
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0146 - val_loss: 0.0156
Model training complete.
```

Step 4 : Evaluate Model Performance

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Use the model.predict() method on X_test to generate predictions.
predicted_values = model.predict(X_test)

# Reshape y_test for plotting if necessary (it's already 1D, but predict output is 2D for single feature)
# We also need to align the length of y_test with predicted_values if some values were skipped due to
look_back

# In our create_sequences function, y was constructed directly from data[i + look_back]
# so y_test corresponds directly to the predictions for X_test

# The model's output `predicted_values` is 2D (num_samples, 1), so flatten it for plotting.
predicted_values = predicted_values.flatten()

# 3. Plot the actual y_test values against the predicted_values.
plt.figure(figsize=(12, 6))
plt.plot(y_test, label='Actual Values', color='blue')
plt.plot(predicted_values, label='Predicted Values', color='red', linestyle='--')
```

```
# 4. Ensure the plot has a title, x-axis label, y-axis label, and a legend.
plt.title('Actual vs. Predicted Time Series Values')
plt.xlabel('Time Step')
plt.ylabel('Value')
plt.legend()
plt.grid(True)
plt.show()

# Optional: Print some performance metrics
from sklearn.metrics import mean_squared_error
rmse = np.sqrt(mean_squared_error(y_test, predicted_values))
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
```

Output :




```
Root Mean Squared Error (RMSE): 0.1251
```

Conclusion :

The LSTM model was successfully implemented and trained to forecast time series data.

Key findings from the experiment include:

1. **High Predictive Accuracy:** The model achieved a Root Mean Squared Error (RMSE) of 0.1251. In time series forecasting, a low RMSE indicates that the predicted values are very close to the actual data points.
2. **Pattern Recognition:** The visualization of "Actual vs. Predicted" values showed that the red dashed line (predictions) closely followed the blue line (actual data). This proves the LSTM successfully learned the seasonal pattern of the sine wave.
3. **Stability of Learning:** Training over 50 epochs allowed the Adam optimizer to minimize the Mean Squared Error effectively. The validation loss remained close to the training loss, suggesting the model did not overfit.
4. **Impact of Look-Back:** Using a look-back window (in this case, 10 steps) provided sufficient historical context for the LSTM to understand the "momentum" of the data and predict the next value with high confidence.